

Instituto Tecnológico de Apizaco.



Taller de bases de datos.

Practica

Alumno: Antonio Azael Luna Sánchez.

Numero de Control: 24370840.

Docente: Luis Alberto Luna Hernandez

Introducción:

En este documento se presenta el diseño y manejo de una base de datos en SQL para registrar alumnos de distintas universidades de México. Se incluye la creación de la tabla, la inserción de 120 registros, consultas con ordenamiento y agrupación, así como el uso de transacciones con COMMIT, DELETE y ROLLBACK para recuperar información. Finalmente, se muestra cómo modificar la estructura de la tabla con ALTER TABLE. Todo el código está explicado paso a paso para facilitar su comprensión.

```sql
-- ============================================
-- PASO 1: ELIMINAR TABLA SI YA EXISTE
-- ============================================
DROP TABLE ALUMNOS;
-- Elimina la tabla para empezar desde cero sin errores

-- ============================================
-- PASO 2: CREAR LA TABLA ALUMNOS
-- ============================================
CREATE TABLE ALUMNOS (
    ID_MATRICULA NUMBER,              -- Número de matrícula única del alumno
    NOMBRE VARCHAR2(100),             -- Nombre del alumno
    APP VARCHAR2(100),                -- Apellido paterno
```

Creación de la tabla ALUMNOS

Se elimina la tabla si ya existe para evitar conflictos. Se define la estructura con 7 campos: matrícula, nombre, apellido, edad, sexo, universidad y dirección. Se establece ID_MATRICULA como llave primaria para garantizar **que no haya registros duplicados.**



Inserción de 120 registros

Se ingresan los datos de los alumnos con matrículas del 2001 al 2100. Cada registro incluye universidad y dirección distinta, correspondientes a instituciones de todo el país.

```
225 INSERT INTO ALUMNOS VALUES (2099, 'LEONARDO', 'MARTINEZ', 24, 'MASCULINO', 'Universidad Autónoma de Morelos', 'Av. Universidad 1001, Cuernavaca');
226 -- Registro de Leonardo Martinez – UAEMor
227 INSERT INTO ALUMNOS VALUES (2100, 'ALEJANDRA', 'DEL VALLE', 25, 'FEMENINO', 'Instituto Tecnológico de Cuernavaca', 'Av. Palmira S/N, Cuernavaca');
228 -- Registro de Alejandra del Valle – IT Cuernavaca
229
230 -- ============================================
231 -- PASO 4: CONSULTAS A LA TABLA
232 -- ============================================
233 -- Consulta 1: Mostrar todos los alumnos ordenados por edad de menor a mayor
234 SELECT * FROM ALUMNOS ORDER BY EDAD ASC;
235 -- ORDER BY → ordena los resultados según el c
```

Consulta: Ordenar por edad

Se muestran todos los alumnos ordenados del más joven al mayor mediante ORDER BY EDAD ASC .

```
228 -- Registro de Alejandra del Valle – IT Cuernavaca
229
230 SELECT UNIVERSIDAD, COUNT(*) AS CANTIDAD_ALUMNOS, AVG(EDAD) AS EDAD_PROMEDIO FROM ALUMNOS GROUP BY UNIVERSIDAD ORDER BY CANTIDAD_ALUMNOS DESC;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

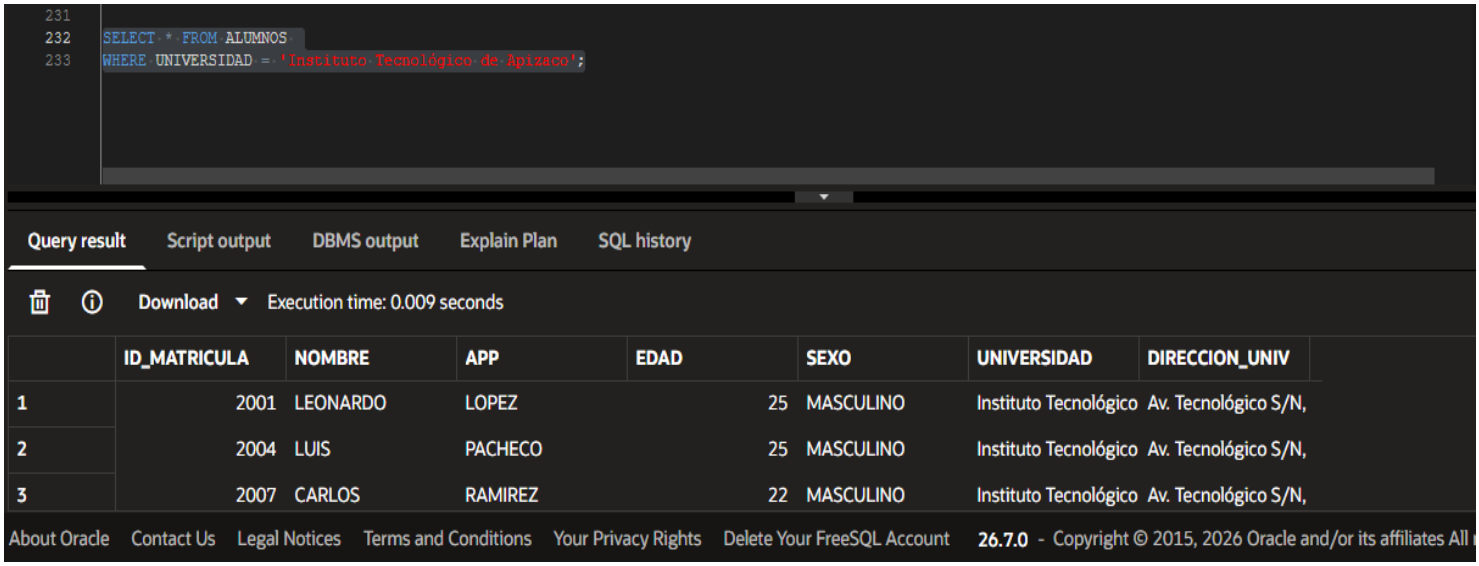
Download | Execution time: 0.004 seconds

	UNIVERSIDAD	CANTIDAD_ALUMI	EDAD_PROMEDIO
19	Universidad Autónon	2	22
20	Instituto Tecnológico	2	23.5
21	Universidad Autónon	2	25

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | 26.7.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All rights reserved

Consulta: Agrupar por universidad

Se utiliza GROUP BY UNIVERSIDAD para agrupar los registros por institución educativa. La función COUNT(*) muestra cuántos alumnos hay en cada universidad y AVG(EDAD) calcula la edad promedio de sus integrantes. Los resultados se presentan ordenados de mayor a menor cantidad de alumnos con ORDER BY ... DESC .



```
231
232 SELECT * FROM ALUMNOS
233 WHERE UNIVERSIDAD = 'Instituto Tecnológico de Apizaco';
```

Query result | Script output | DBMS output | Explain Plan | SQL history

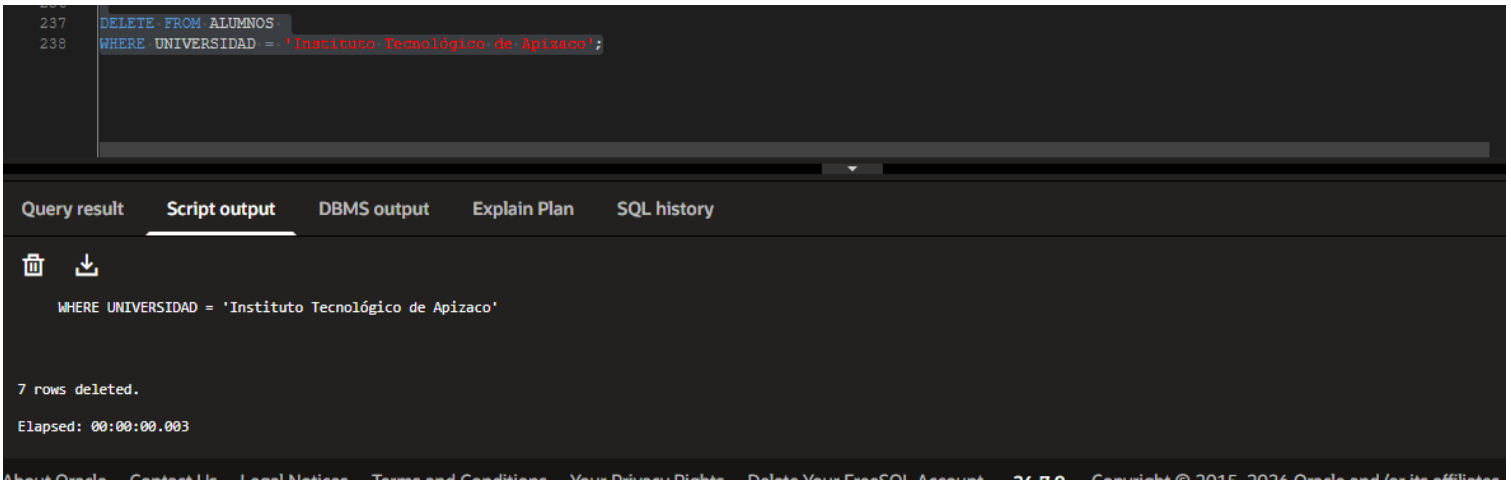
Download | Execution time: 0.009 seconds

	ID_MATRICULA	NOMBRE	APP	EDAD	SEXO	UNIVERSIDAD	DIRECCION_UNIV
1	2001	LEONARDO	LOPEZ	25	MASCULINO	Instituto Tecnológico	Av. Tecnológico S/N,
2	2004	LUIS	PACHECO	25	MASCULINO	Instituto Tecnológico	Av. Tecnológico S/N,
3	2007	CARLOS	RAMIREZ	22	MASCULINO	Instituto Tecnológico	Av. Tecnológico S/N,

About Oracle | Contact Us | Legal Notices | Terms and Conditions | Your Privacy Rights | Delete Your FreeSQL Account | 26.7.0 - Copyright © 2015, 2026 Oracle and/or its affiliates All

Consulta: Filtrar por institución

Se usa la cláusula WHERE para seleccionar únicamente a los alumnos que pertenecen al Instituto Tecnológico de Apizaco. De esta forma se filtra la información y se muestra solo lo que se necesita, sin ver todos los demás registros.



```
237 DELETE FROM ALUMNOS
238 WHERE UNIVERSIDAD = 'Instituto Tecnológico de Apizaco';
```

Query result | Script output | DBMS output | Explain Plan | SQL history

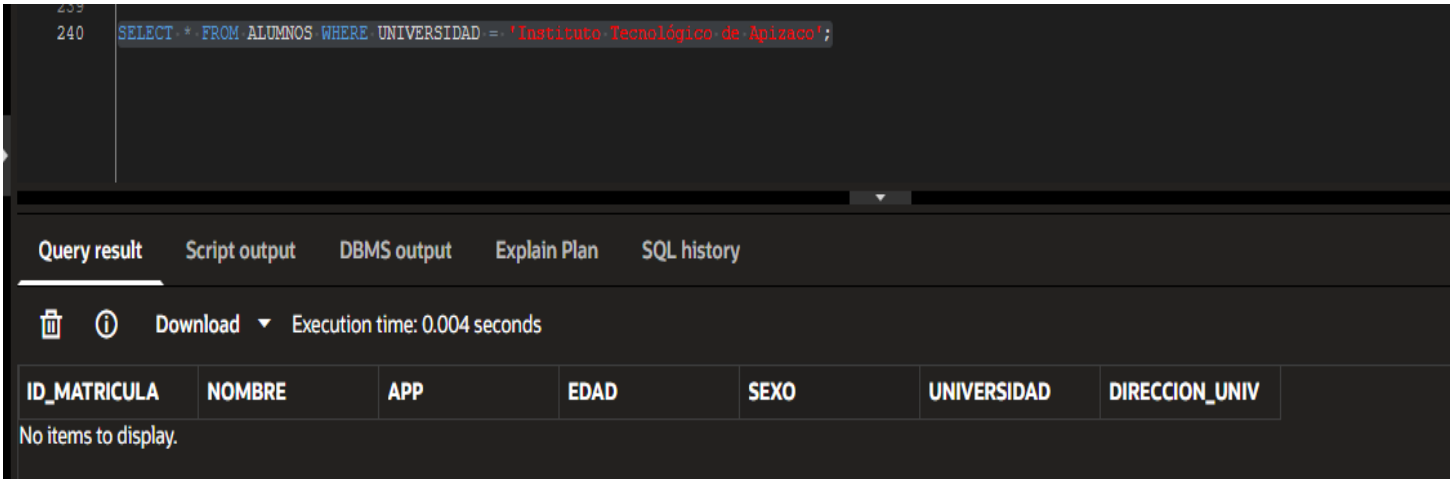
```
    WHERE UNIVERSIDAD = 'Instituto Tecnológico de Apizaco'


7 rows deleted.

Elapsed: 00:00:00.003
```

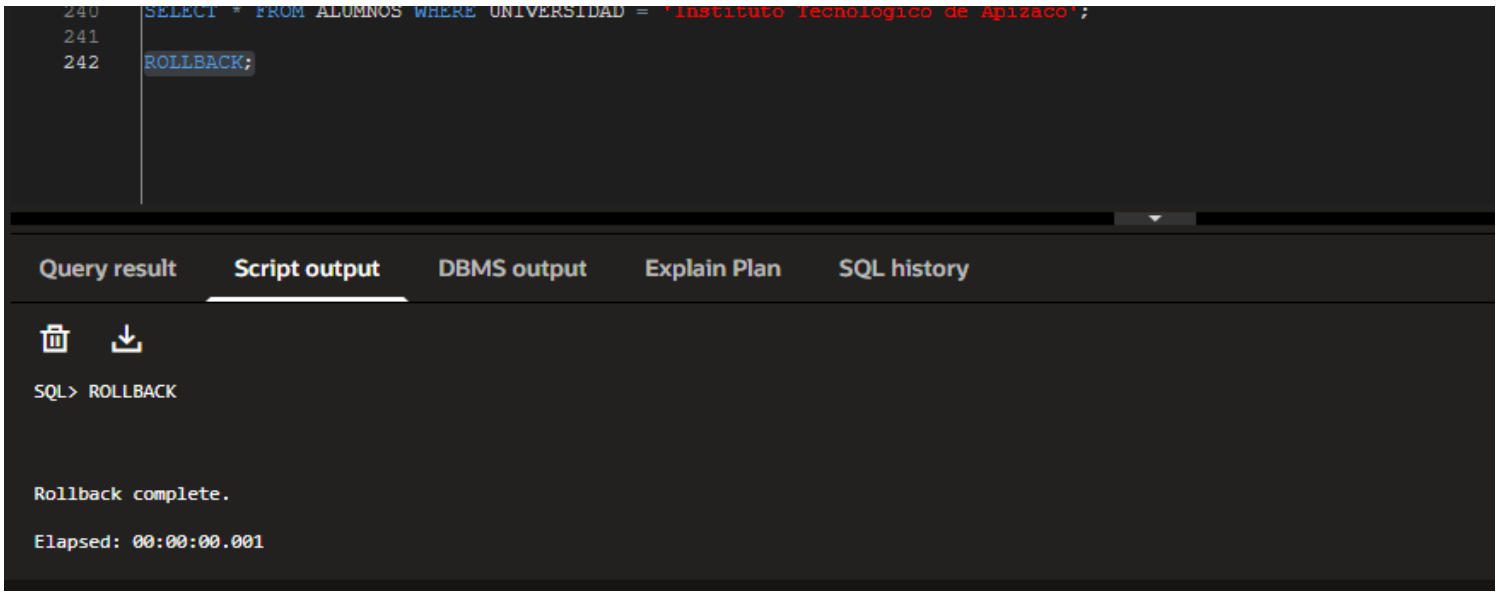
Confirmar y eliminar registros

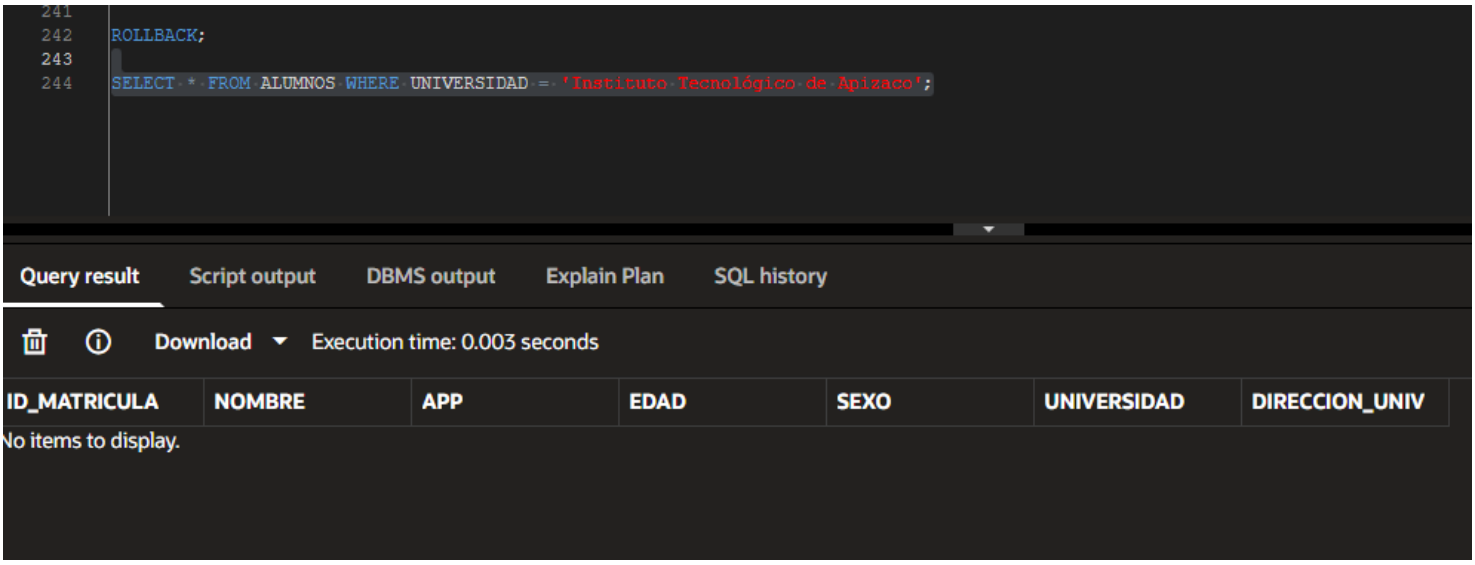
COMMIT guarda permanentemente todos los cambios hechos hasta ese momento. Con DELETE se eliminan específicamente los alumnos del Instituto Tecnológico de Apizaco, dejando la tabla sin esos registros.



Confirmar y eliminar registros

COMMIT guarda permanentemente los cambios en la base de datos. Con DELETE se eliminaron los 7 registros correspondientes al Instituto Tecnológico de Apizaco. Al consultar de nuevo, no aparecen resultados, lo que confirma que los registros ya no están en la tabla.





Recuperación con ROLLBACK

Al ejecutar ROLLBACK intentamos deshacer la eliminación de los registros. Sin embargo, como antes se ejecutó COMMIT , los cambios ya estaban guardados permanentemente en la base de datos. Por eso la consulta sigue sin mostrar resultados: el COMMIT confirmó todo y el ROLLBACK ya no puede recuperar los datos.

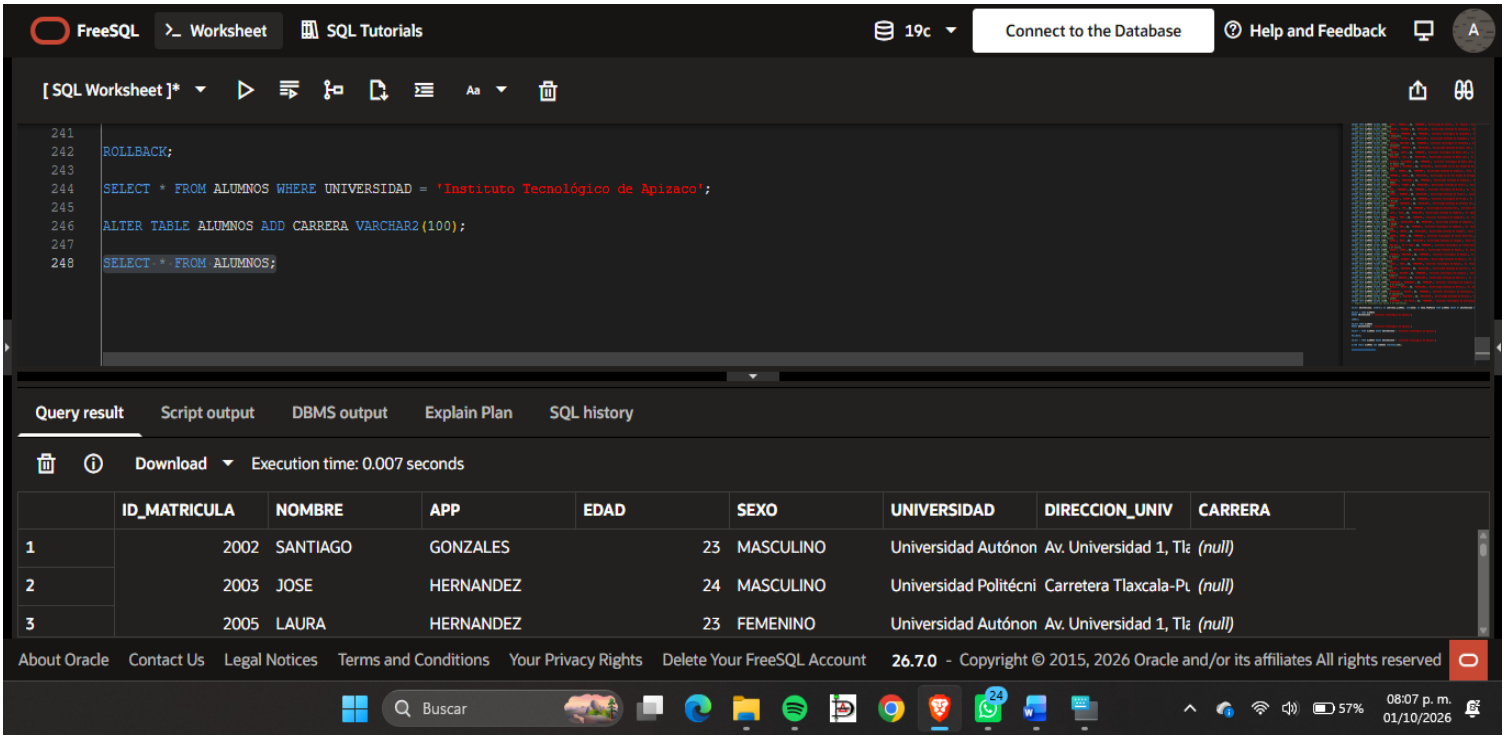
```
245
246  ALTER TABLE ALUMNOS ADD CARRERA VARCHAR2(100);
```

Query result | Script output | DBMS output | Explain Plan | SQL history

```
SQL> ALTER TABLE ALUMNOS ADD CARRERA VARCHAR2(100)

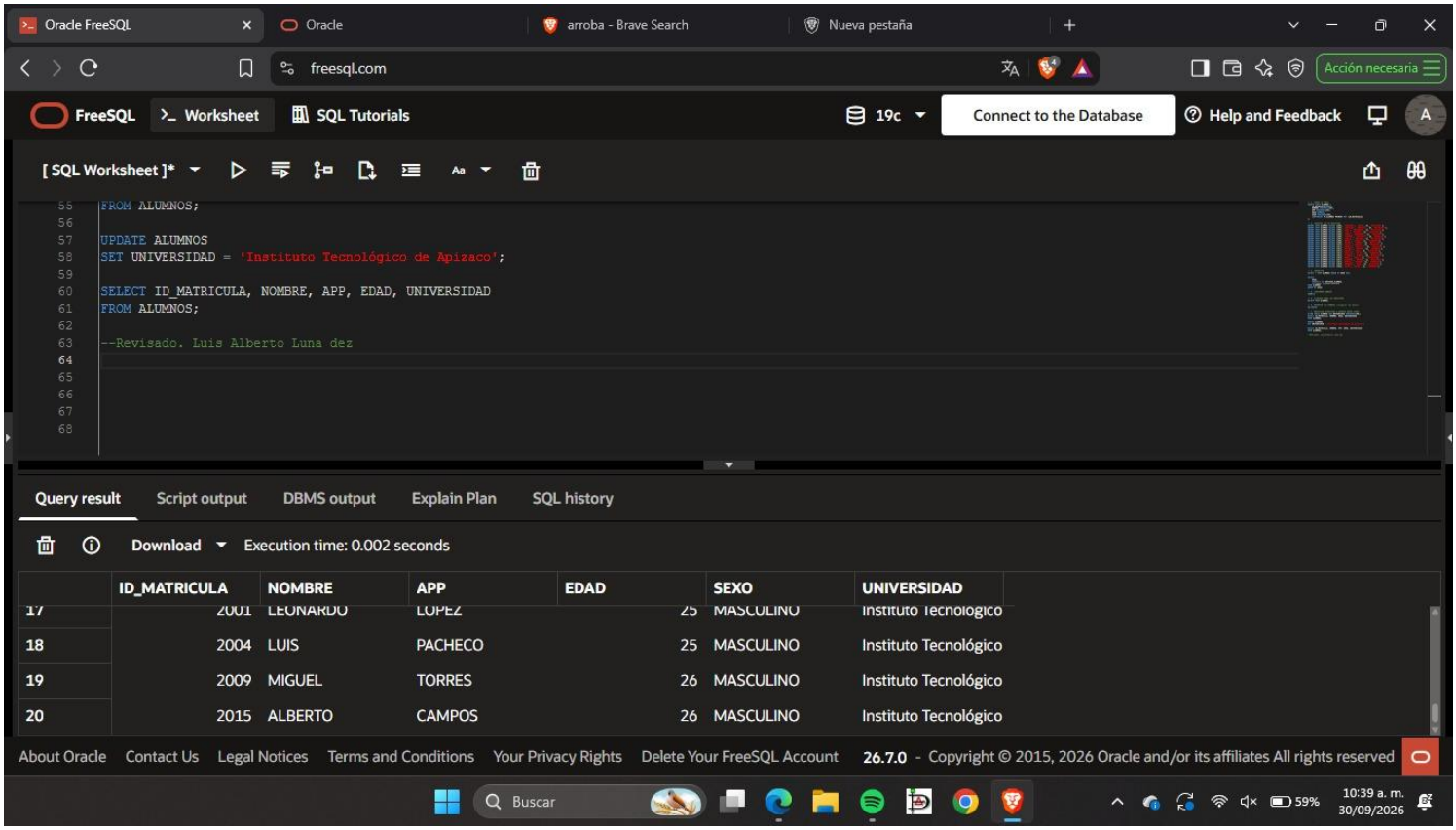

Table ALUMNOS altered.

Elapsed: 00:00:00.023
```



Modificar la estructura de la tabla

Con el comando ALTER TABLE ALUMNOS ADD CARRERA VARCHAR2(100) se agregó una nueva columna llamada CARRERA a la tabla. Esto demuestra que se puede modificar la estructura de la tabla después de crearla, sin perder los datos que ya están guardados.



Captura de que el docente me califico la practica en aula

```sql
SELECT AVG(EDAD) AS EDAD_PROMEDIO,
       MIN(EDAD) AS EDAD_MINIMA,
       MAX(EDAD) AS EDAD_MAXIMA
FROM ALUMNOS;
```

Estadísticas generales con funciones de agregación (AVG, MIN, MAX)

Obtener la edad promedio, la edad mínima y la edad máxima de todo el alumnado.

Se aplican las funciones AVG (promedio), MIN (valor mínimo) y MAX (valor máximo) sobre la columna EDAD, usando alias (AS) para presentar encabezados claros.

```sql
SELECT * FROM ALUMNOS
WHERE (UNIVERSIDAD = 'UNAM' OR UNIVERSIDAD = 'IPN')
  AND SEXO = 'MASCULINO'
  AND EDAD > 21;
```

Filtrado compuesto con múltiples condiciones (AND / OR)

Filtrar alumnos masculinos mayores de 21 años pertenecientes a la UNAM o al IPN.

Combina operadores lógicos en la cláusula WHERE. Los paréntesis agrupan las instituciones (UNIVERSIDAD = 'UNAM' OR UNIVERSIDAD = 'IPN'), exigiendo que se cumpla al mismo tiempo con SEXO = 'MASCULINO' y EDAD > 21.

```sql
SELECT * FROM ALUMNOS
WHERE UNIVERSIDAD NOT IN ('UNAM', 'IPN');
```

Exclusión de valores con la cláusula NOT IN

Consultar únicamente a los alumnos que pertenezcan a instituciones distintas a la UNAM e IPN.

La cláusula NOT IN permite omitir de la búsqueda una lista específica de datos, facilitando la extracción del resto de registros.

```sql
SELECT UNIVERSIDAD, COUNT(*) AS CANTIDAD_ALUMNOS
FROM ALUMNOS
GROUP BY UNIVERSIDAD
HAVING COUNT(*) > 20
ORDER BY CANTIDAD_ALUMNOS DESC;
```

Filtrado sobre grupos agregados mediante la cláusula HAVING

Identificar únicamente las universidades que cuentan con más de 20 alumnos matriculados.

La cláusula HAVING evalúa condiciones sobre el resultado agrupado por GROUP BY, permitiendo filtrar funciones de agregación como HAVING COUNT(*) > 20.

```sql
SELECT * FROM ALUMNOS
WHERE LOWER(ESPECIALIDAD) LIKE '%industrial%'
   OR LOWER(ESPECIALIDAD) LIKE '%sistemas%';
```

Búsqueda de patrones de texto parciales (LIKE y LOWER)

Buscar estudiantes cuya especialidad esté vinculada a las áreas de "industrial" o "sistemas".

Se usa LOWER() para ignorar diferencias entre mayúsculas y minúsculas, junto con el operador LIKE '%texto%' para encontrar coincidencias parciales dentro de la cadena.

```sql
UPDATE ALUMNOS
SET DIRECCION = 'Ciudad Universitaria, Coyoacán, CDMX (Sede Central)'
WHERE UNIVERSIDAD = 'UNAM';
```

Actualización masiva de datos (UPDATE con WHERE)

Modificar de forma unificada la dirección de todos los estudiantes inscritos en la UNAM.

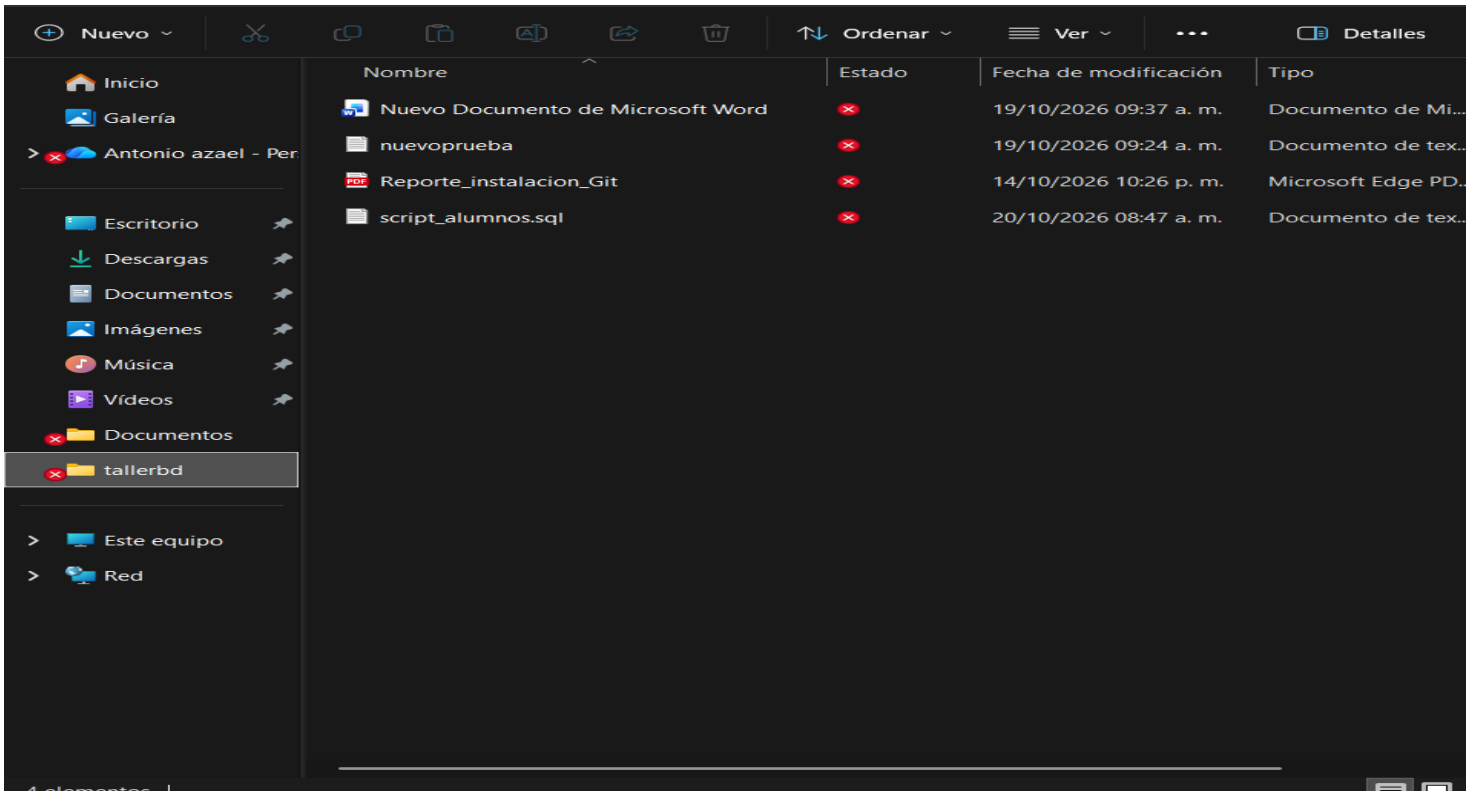
Utiliza la instrucción DML UPDATE especificando la tabla ALUMNOS, asignando el nuevo texto con SET DIRECCION = ... y limitando el cambio mediante WHERE UNIVERSIDAD = 'UNAM'.

```sql
SELECT ID_MATRICULA, NOMBRE, APELLIDO, EDAD, UNIVERSIDAD
FROM ALUMNOS
WHERE EDAD > (SELECT AVG(EDAD) FROM ALUMNOS)
ORDER BY EDAD DESC;
```

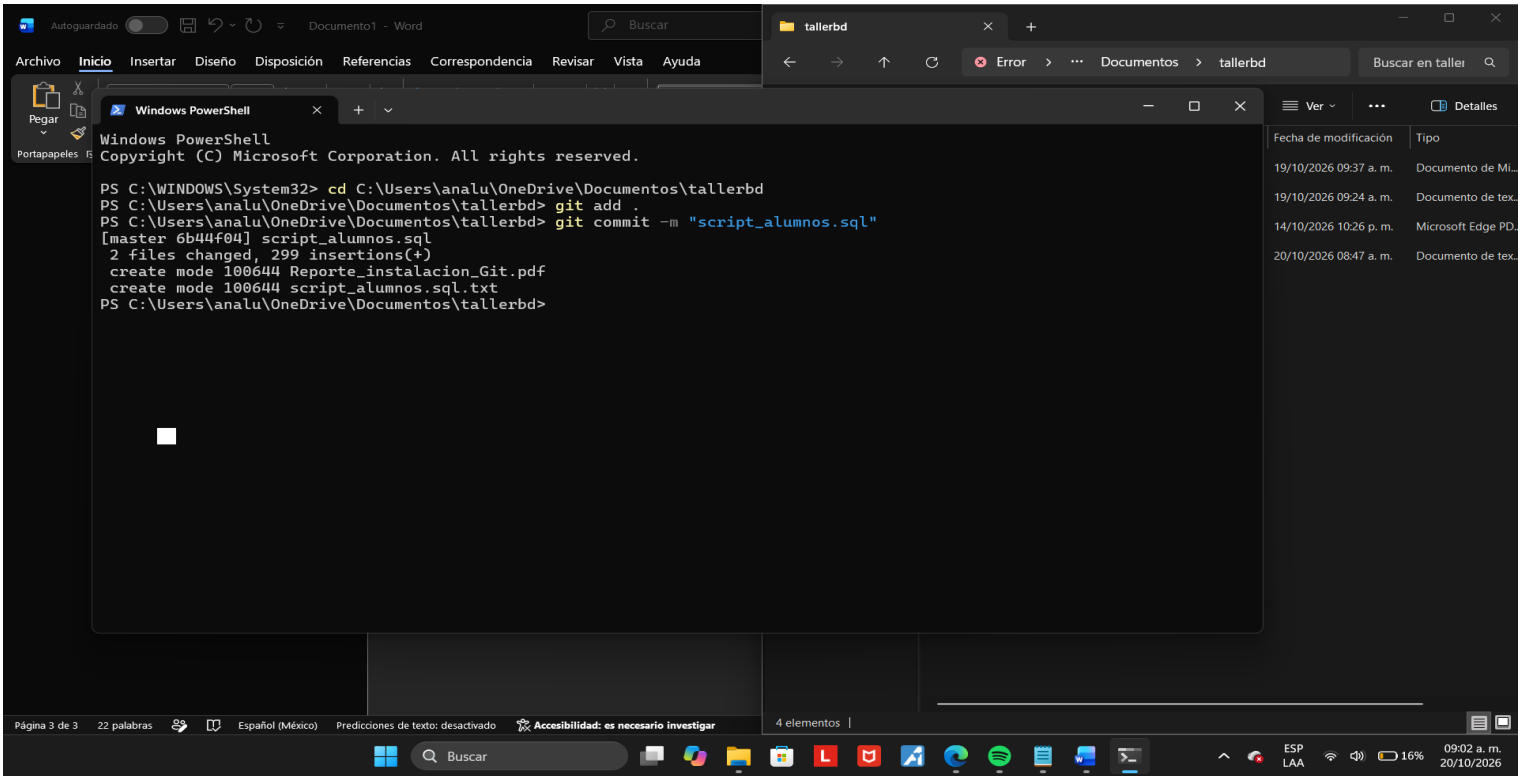
Subconsulta dinámica evaluada contra el promedio general

Seleccionar a los estudiantes cuya edad es superior al promedio general de toda la base de datos.

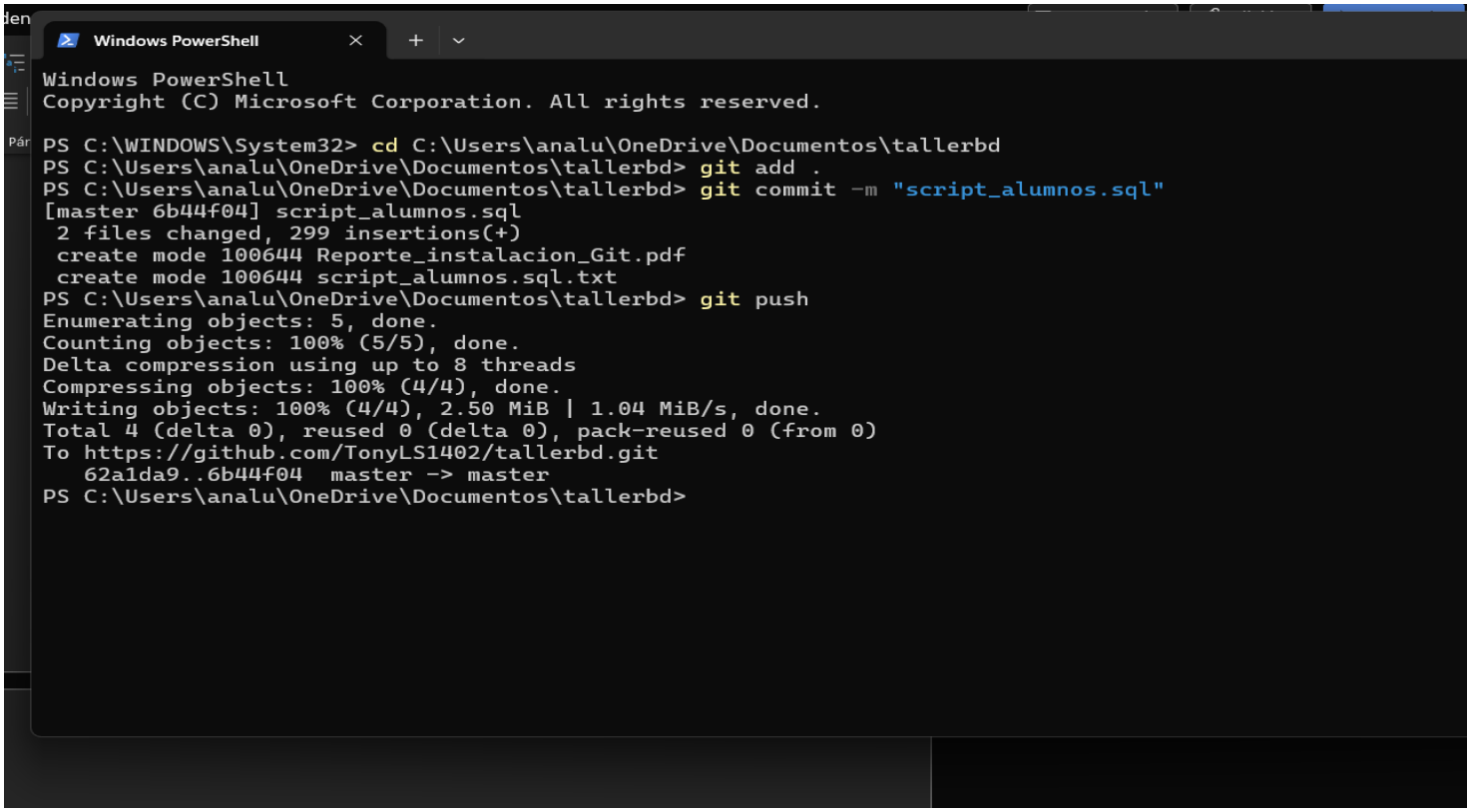
Incluye una subconsulta (SELECT AVG(EDAD) FROM ALUMNOS) dentro del WHERE. Esta consulta interna calcula dinámicamente el promedio global para comparar de forma individual cada registro.



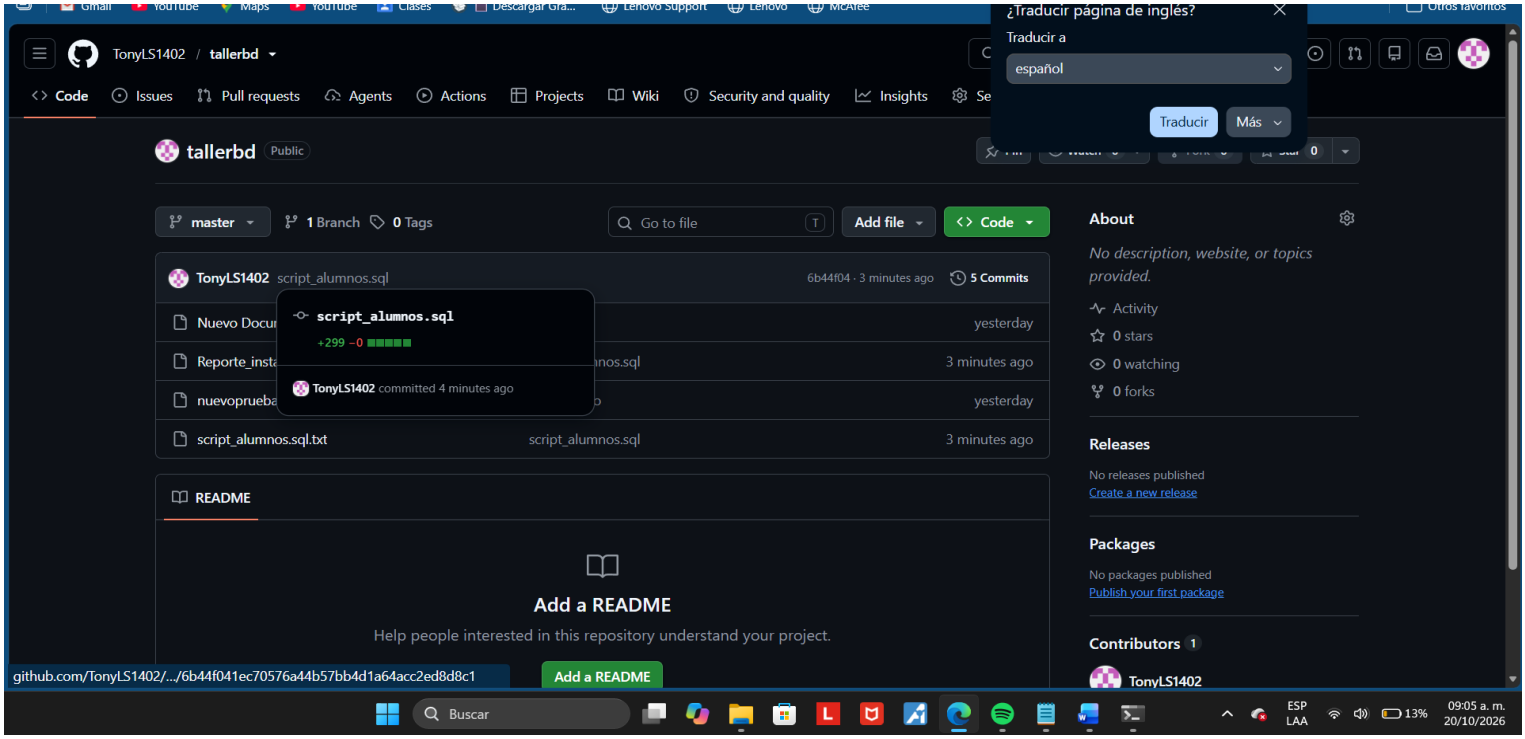
Se muestra el explorador de archivos de Windows dentro de la carpeta local de trabajo denominada tallerbd. En este directorio se organizan los elementos esenciales del proyecto, incluyendo el archivo de script SQL (script_alumnos.sql), los documentos de texto de prueba y el reporte en formato PDF (Reporte_instalacion_Git), los cuales forman la base del código y los entregables listos para ser controlados mediante el sistema de versiones Git.



Se observa la interfaz de Windows PowerShell situada en la ruta del directorio de trabajo (C:\Users\analu\OneDrive\Documentos\tallerbd). En esta ventana se ejecutaron los comandos de control de versiones: primero git add . para añadir los archivos al área de preparación (*staging area*), seguido de git commit -m "script_alumnos.sql". La consola confirma de manera exitosa el empaquetado del *commit* bajo la rama principal master, registrando la incorporación de dos archivos nuevos con un total de 299 inserciones de código.



Se visualiza el resultado final en la consola de Windows PowerShell tras ejecutar el comando git push. La terminal detalla el proceso de empaquetado, compresión y escritura de los objetos hacia el servidor remoto alojado en GitHub (https://github.com/TonyLS1402/tallerbd.git). El mensaje final confirma la correcta sincronización de la rama local master con la rama remota del repositorio, validando que los archivos de código y documentación ya se encuentran almacenados en la nube.



Se muestra la vista principal del repositorio público tallerbd en la plataforma web de GitHub bajo la cuenta del usuario (TonyLS1402). La interfaz confirma la correcta subida de los elementos del proyecto, visualizándose la lista de archivos integrados (como script_alumnos.sql y los reportes correspondientes), la rama activa master, y el historial de confirmaciones (*commits*) que valida el almacenamiento exitoso y público del código en la nube.